# Section 12

Systems Programming ECE3502

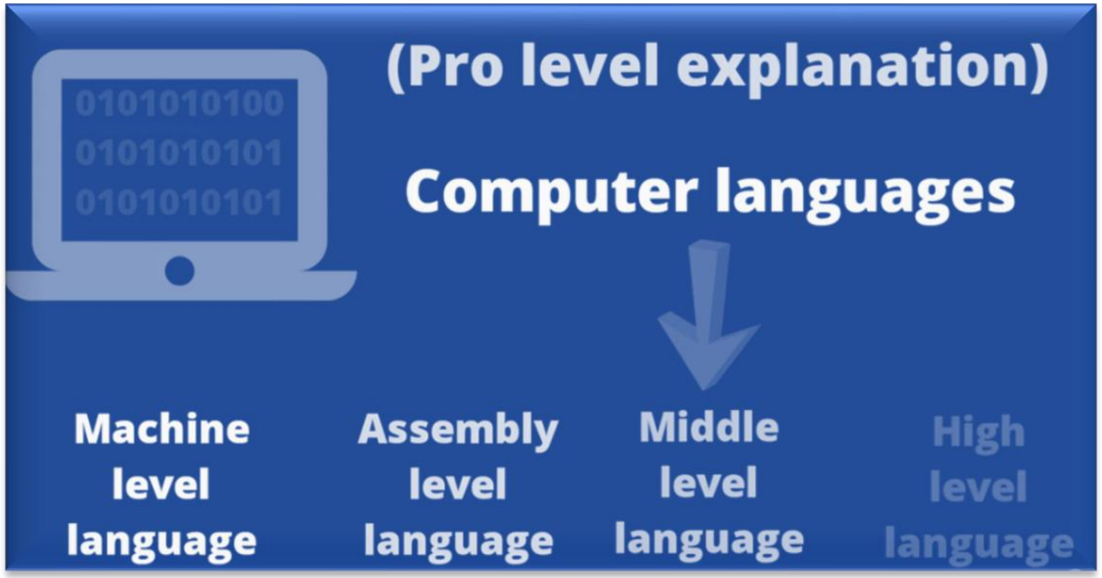
# Translators

- A translator or programming language processor that converts a computer program from language to another.

(convert the programming language into a machine language)

- It takes a program written in source code and converts into machine code.

- It discovers and identifies the error during translation.



# Types of Translators

**Assembler:** the assembler is used to translate the program written in assembly language into machine level language.

**Compilers:** the compiler will read the complete source program written in high level language, if the code is error free then it will convert into machine level language.

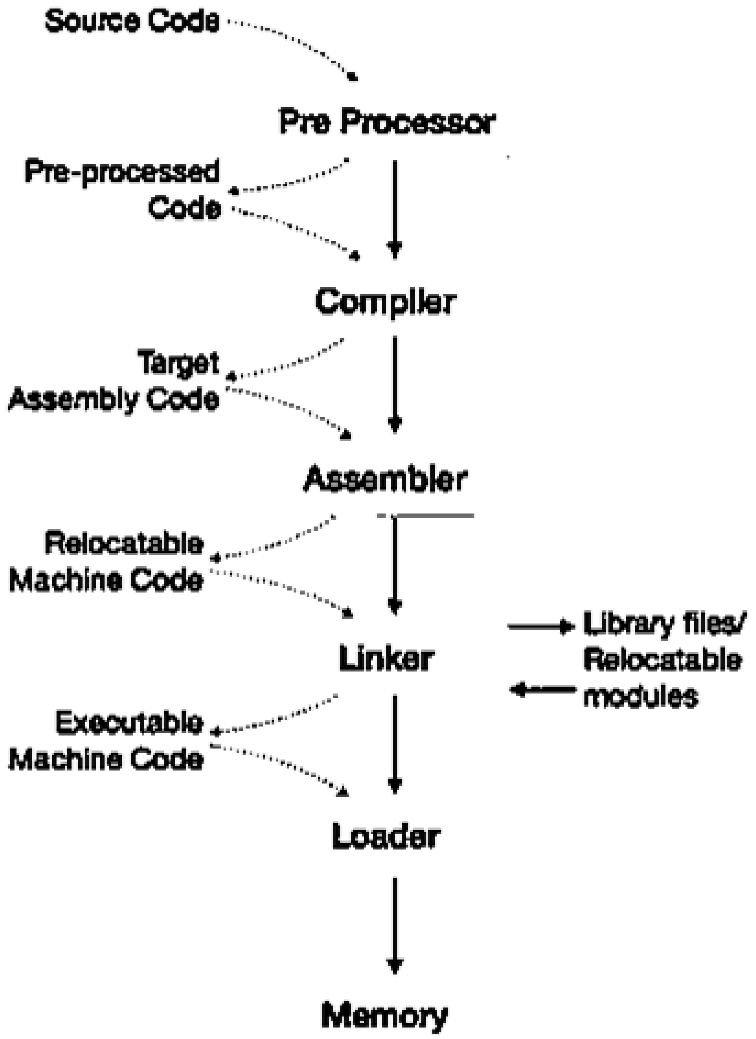
A = B + C * D → Compiler →

```
LOD R1,C        // Load the value of C into reg 1
MUL R1,D        // Multiply the value of D by reg 1
STO R1,TEMP1    // Store the result in TEMP1
LOD R1,B        // Load the value of B into reg 1
ADD R1,TEMP1    // Add value of Temp1 to register 1
STO R1,TEMP2    // Store the result in TEMP2
MOV A,TEMP2     // Move TEMP2 to A, the final result
```

**(High level language)** **(Translator)** **(Low level language)**

**Interpreter:** just like compiler, it convert one at a time and reports errors detect once, while doing conversion (faster than a compiler).

# Translators



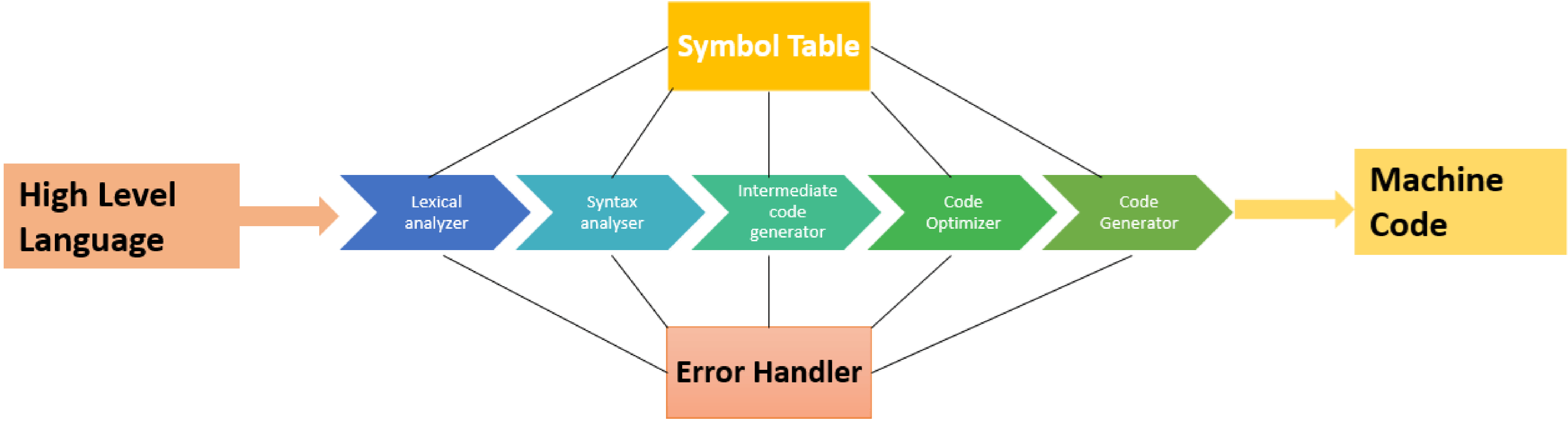
# The Structure of a Compiler

## Analysis (FRONT END COMPILER):

- breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- The grammatical structure is used to create an intermediate representation.
- The analysis part produces informative messages if the source program is syntactically ill formed or semantically unsound.

## Synthesis(BACK ENDCOMPILER):

- Constructs the desired target program from the intermediate representation and the information in the symbol table



# Definition of Grammars

**A context-free grammar has four components:**

1. A set of **terminal symbols**, sometimes referred to as **"tokens"**.
2. A set of **nonterminals**, sometimes called **"syntactic variables"**. Each nonterminal represents a set of strings of terminals.
3. A designation of one of the non terminals as the **start symbol**
4. A set of **productions**, where each production consists of
   a. a non terminal called the head or left side of the production.
   b. an arrow, (sometimes ::=)
   c. A sequence of terminals and/or no

## Terminal Symbols

a. Lowercase letters early in the alphabet, such as a , b , c.
b. Operator symbols such as +, * , and so on.
c. Punctuation symbols such as parentheses, comma, and so on.
d. The digits 0 , 1 , . . ., 9.
e. Boldface strings such as id or if, each of which represents a single terminal symbol.

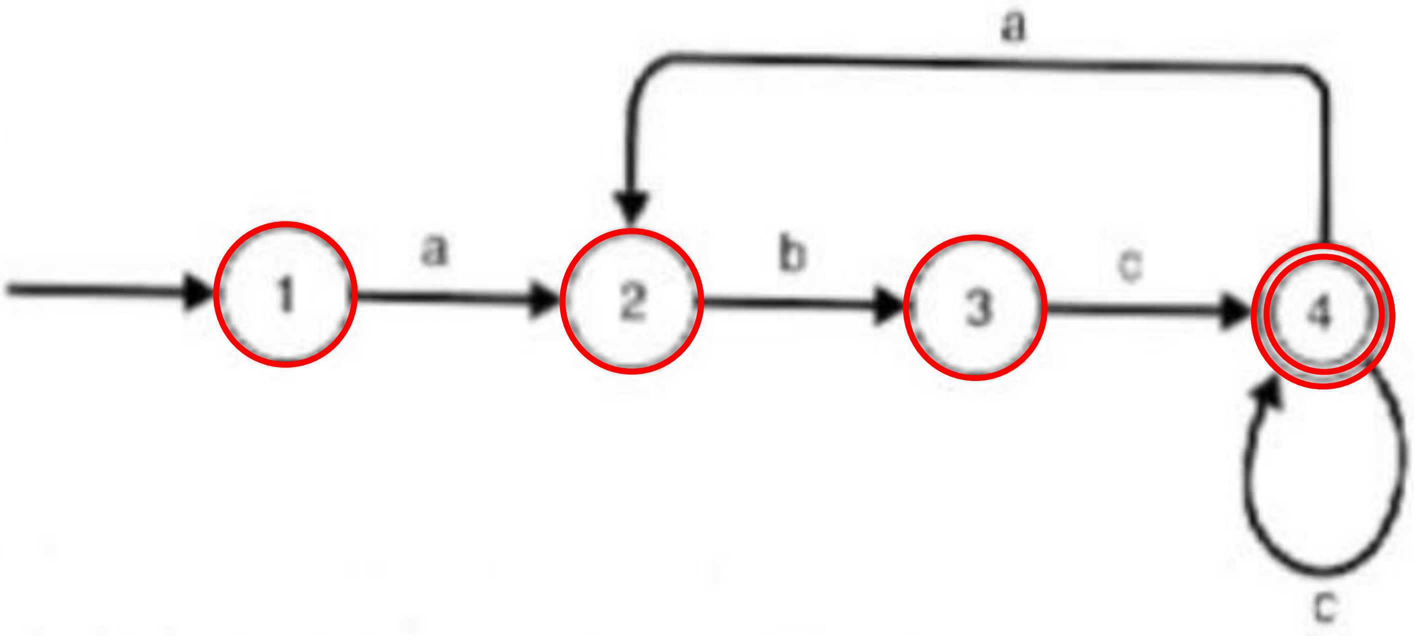
## Non Terminal Symbols

a. Uppercase letters early in the alphabet, such as A , B , C .
b. The letter S , which, when it appears, is usually the start symbol.
c. Lowercase, italic names such as expr or stmt.
d. When discussing programming constructs, uppercase letters may be used to represent non terminals for the constructs. For example, nonterminals for expressions, terms, and factors are often represented by E, T, and F, respectively.

# Finite Automata (FA)

Indicate if the scanner identifies the following statements

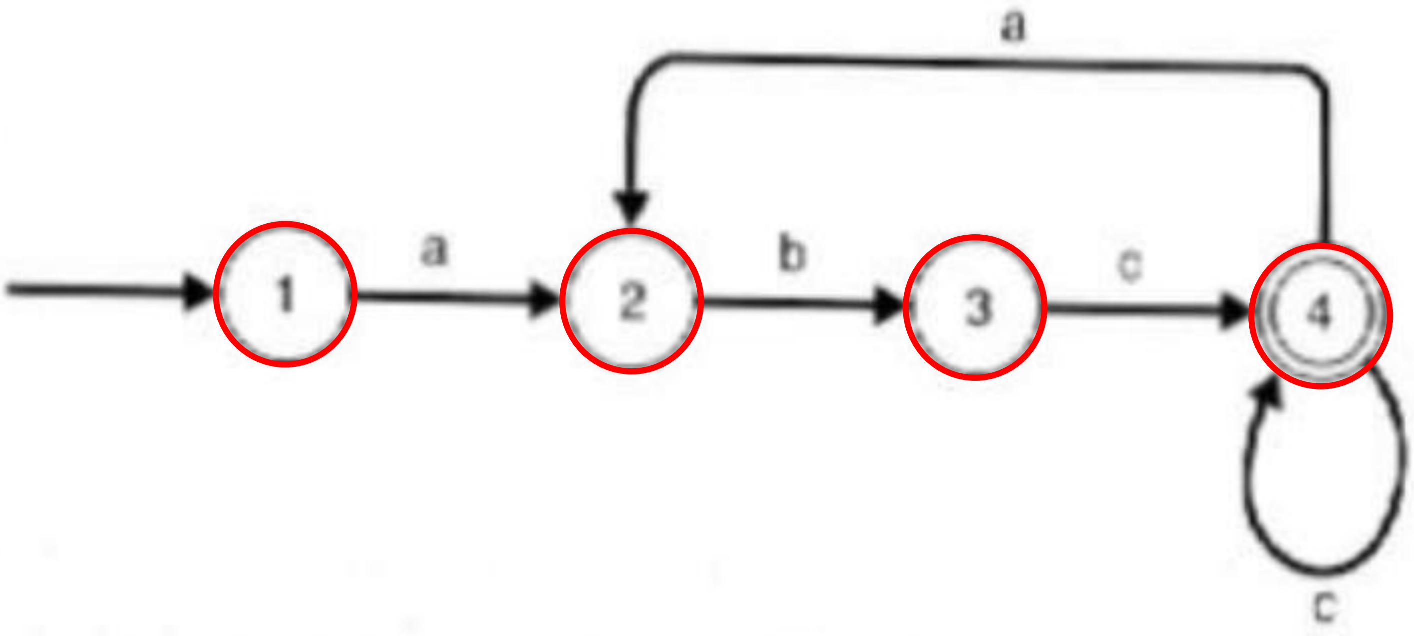
a) abcc ✓
b) abcb
c) abca
d) abccb
e) abcabcc
f) ab



# Finite Automata (FA)

Indicate if the scanner identifies the following statements

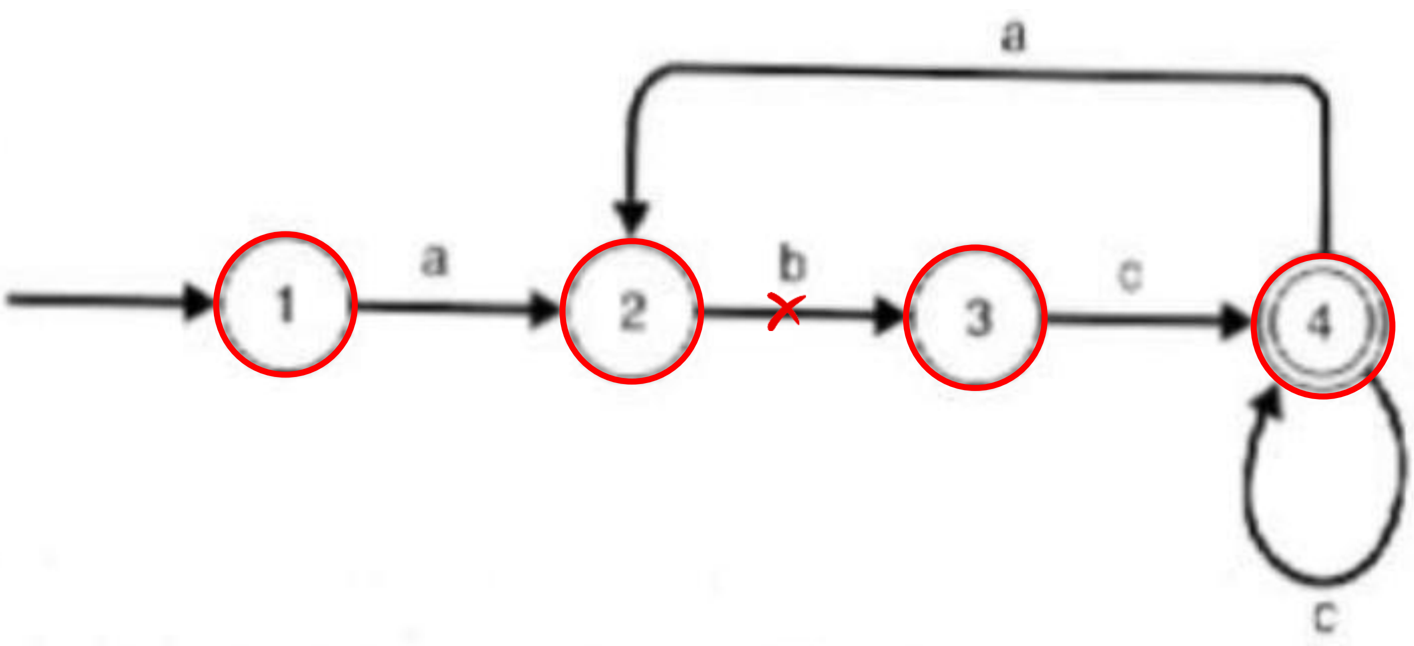
a) abcc ✓
b) abcb ✗
c) abca
d) abccb
e) abcabcc
f) ab



# Finite Automata (FA)

Indicate if the scanner identifies the following statements

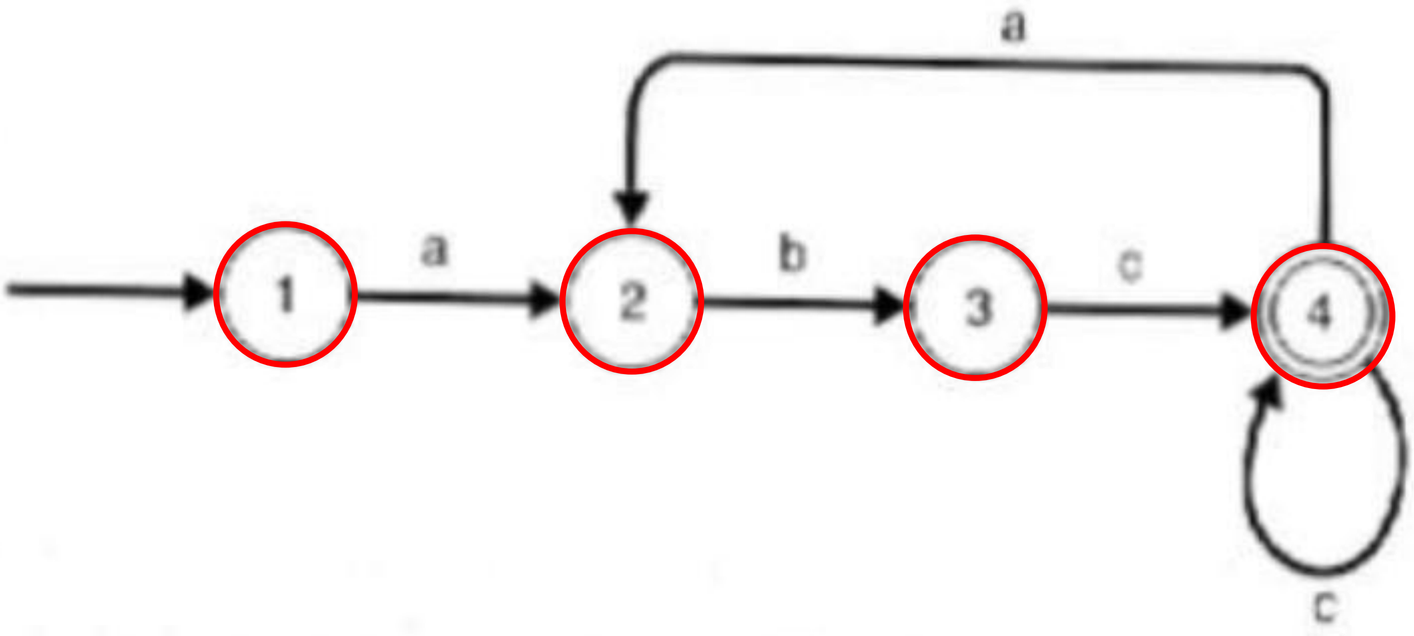
a) abcc ✓
b) abcb ✗
c) abca ✗
d) abccb
e) abcabcc
f) ab



# Finite Automata (FA)

Indicate if the scanner identifies the following statements

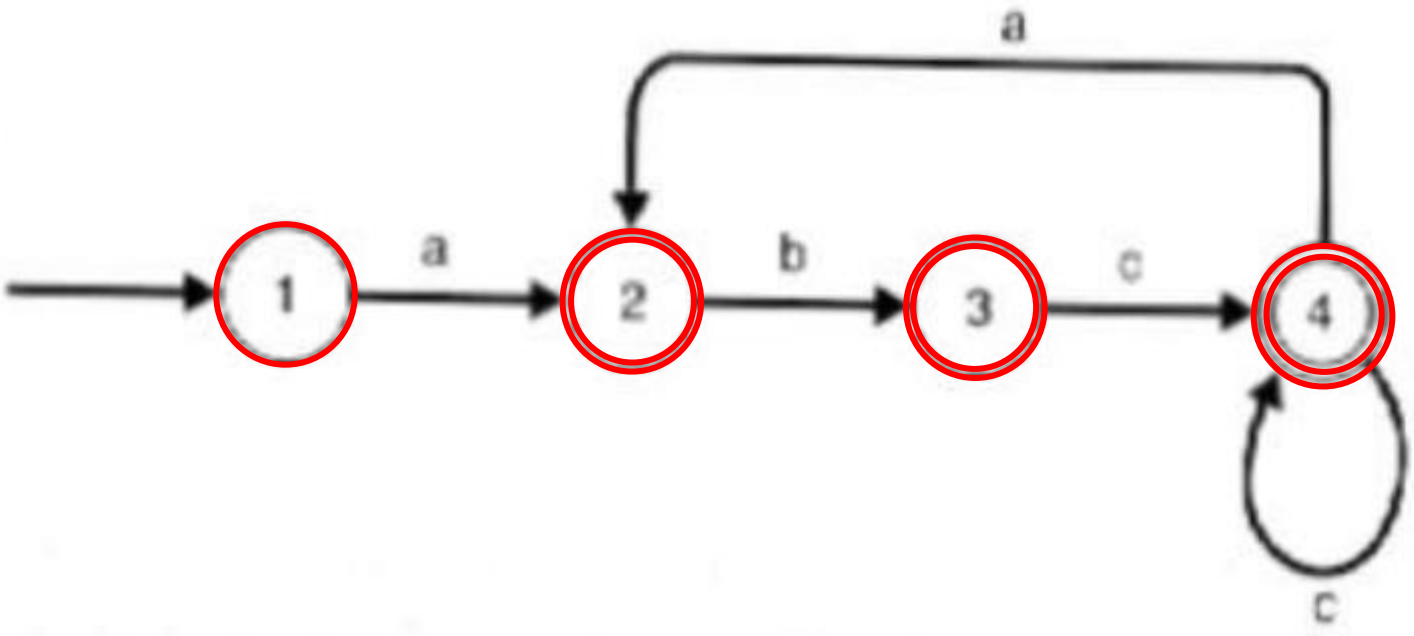
a) abcc ✓
b) abcb ✗
c) abca ✗
d) abccb ✗
e) abcabcc
f) ab



# Finite Automata (FA)

Indicate if the scanner identifies the following statements

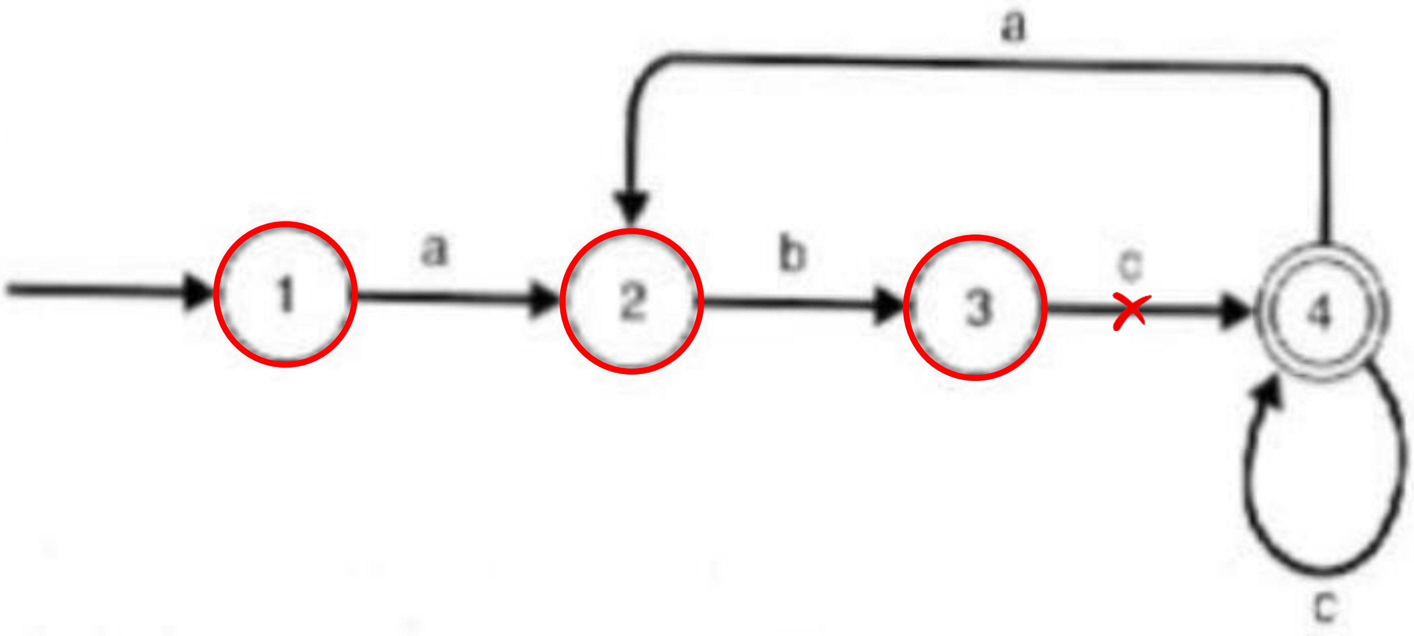
a) abcc ✓
b) abcb ✗
c) abca ✗
d) abccb ✗
e) abcabcc ✓
f) ab



# Finite Automata (FA)

Indicate if the scanner identifies the following statements

a) abcc ✓
b) abcb ✗
c) abca ✗
d) abccb ✗
e) abcabcc ✓
f) ab ✗



# Top-Down Approach Syntax Analysis

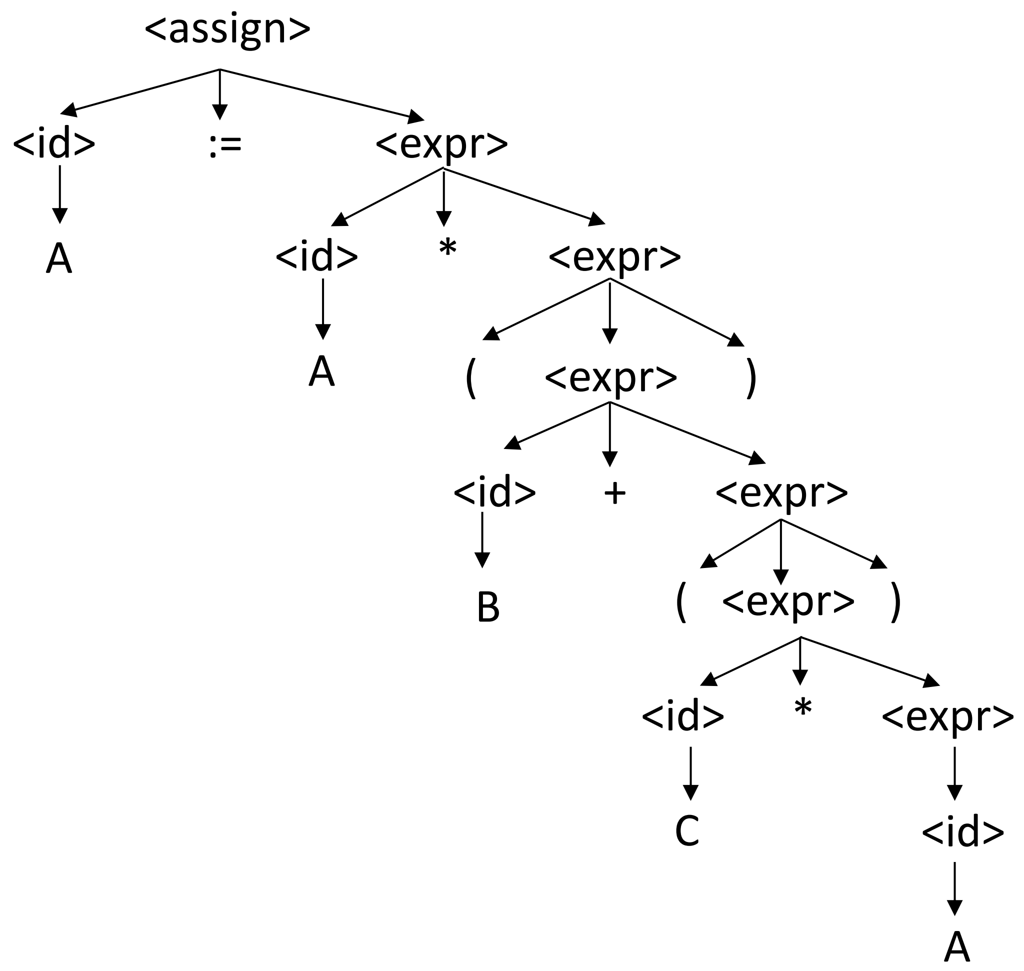
1. Draw the parse trees for the following statements using grammar 1

**Grammar 1**

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
| <id> * <expr>
| ( <expr> )
| <id>

a. A:= A * ( B + ( C * A ))

<assign> → <id> := <expr>
A := <expr>
A := <id> * <expr>
A := A * (<expr>)
A := A * (<id> + <expr>)
A := A * (B + <expr>)
A := A * (B + (<expr>))
A := A * (B + (<id> * <expr>))
A := A * (B + (C * <expr>))
A := A * (B + (C * <id>))
A := A * (B + (C * A))



Draw the parse trees for the following statements using grammar 2:

**Grammar 2**

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → ( <expr> )
| <id>

# a. A := ( A + B ) * C

<assign> → <id> := <expr>

A := <expr>

A := <term>

A := <term> * <factor>

A := <factor> * <factor>

A := (<expr>) * <factor>

A := (<expr> + <term>) * <factor>

A := (<term> + <term>) * <factor>

A := (<factor> + <term>) * <factor>

A := (<id> + <term>) * <factor>

A := (A + <term>) * <factor>

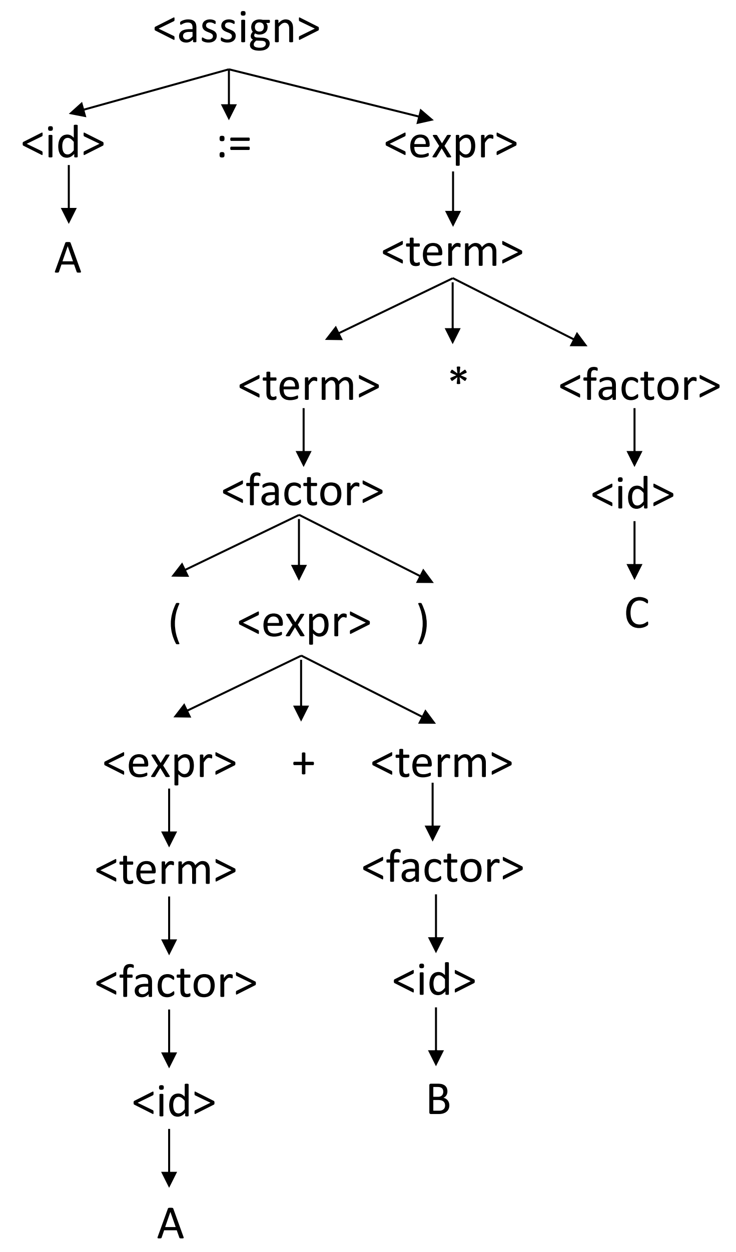
A := (A + <factor>) * <factor>

A := (A + <id>) * <factor>

A := (A + B) * <factor>

A := (A + B) * <id>

A := (A + B) * C

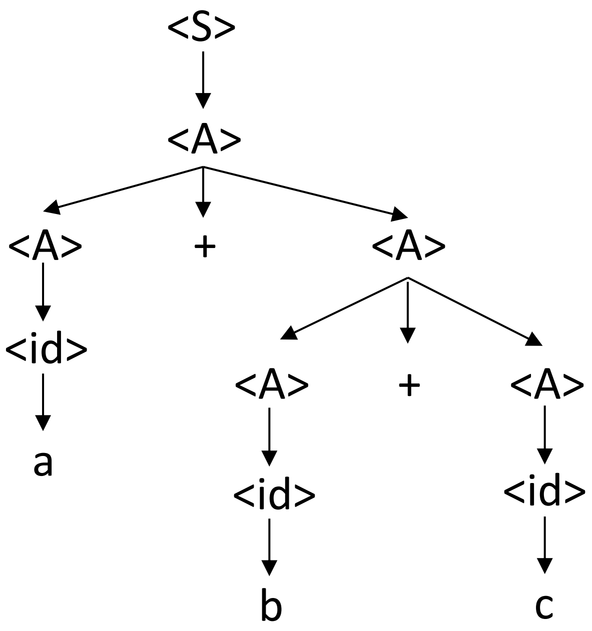


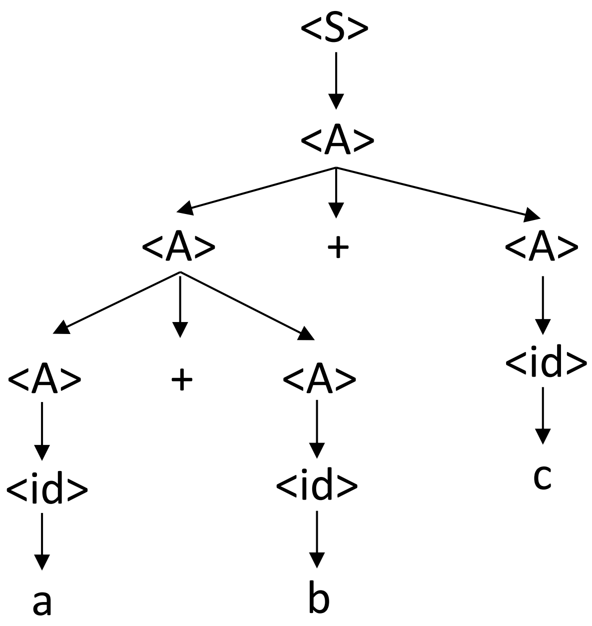
# Top-Down Approach Syntax Analysis

3) Prove that grammar 3 is ambiguous.

**Grammar 3**

<S> ➔ <A>
<A> ➔ <A> + <A> | <id>
<id> ➔ a | b | c



# Top-Down Approach Syntax Analysis

4) Consider grammar 4. Which of the following sentences are in the language generated by this grammar?

**<u>Grammar 4</u>**

<S> → <A> a <B> b
<A> → <A> b | b
<B> → a <B> | a

a. baab
b. bbbab
c. bbaaaaa
d. bbaab

Solution:
prof a:

<S> → <A> a <B> b
<S> → b a <B> b
<S> → b a a b ✓

Solution:
prof b:

<S> → <A> a <B> b
<S> → <A> b a <B> b
<S> → <A> b b a <B> b
<S> → b b b a <B> b
<S> → b b b a a b ✗

# Top-Down Approach Syntax Analysis

5) Consider grammar 5.Which of the following sentences are in the language generated by this grammar?

**Grammar 5**

<S> → a <S> c <B> | <A> | b
<A> → c <A> | c
<B> → d | <A>

a. abcd
b. acccbd
c. acccbcc
d. acd
e. accc

Solution:
prof a:

<S> → a <S> c <B>
<S> → a b c <B>
<S> → abcd ✓

Solution:
prof b:

<S> → a <S> c <B>
<S> → a <A> c <B>
<S> → a c <A> c <B>
<S> → a c c c <B>
<S> → a c c c d ✗